

Compiling Optimal Retry Policies with Charged Atomic Calls

ANONYMOUS AUTHOR(S)

We synthesize causal retry policies for dependent transformations under a finite interference allowance and declared resource prices. Nonzero validation and callback charges can make a third mode essential: prepare outside protection, then atomically reuse or recompute the result. A potential proves that three serial modes attain the minimum worst cost of an atomic-call interface, including programs retaining several preparations and changing jobs between calls. For identity-based Java callbacks, a matching adversary uses at most B actual external writes. All-budget values have supporting slopes in $\{0\} \cup \{w_i + \min(v_i, k_i), w_i + p_i\}$, at most $4n$ positive marginal runs per state, and $2^n \text{poly}(L)$ construction/checking for binary input length L . A mixed-price class admits exact $O(|E| + n \log(n + 1))$ construction and checking through a two-fee reduction; a general zero-fee subcase is NP-complete at $B = 1$. We check 19,485 compact/fallback inputs and 3,700 native Java executions across two implementations. Compact construction/checking completes 32,768-job inputs with 65,536 root runs in at most 1.604 seconds. Requested-value methods can be cheaper. A Deephaven integration loads profiles and preserves captured directives while live actions change under a caller-owned update bound. A separate 72-cell API calibration fails to establish measured third-mode benefit. The guarantees concern the stated resource contracts; application prevalence and elapsed-time improvement remain unestablished.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; *Concurrent programming structures*.

Additional Key Words and Phrases: program synthesis, concurrency, retries, quantitative games, resource guarantees

1 Introduction

A program can transform a shared object by computing a tentative result and validating it, or by computing inside an atomic callback. The first choice risks repeated work; the second pays for protection. Dependencies add another decision: which ready transformation to attempt next. A compiler for this problem should account for the available operations, rather than assume that a chosen retry pattern already contains an optimal program.

Call costs change that pattern. Consider two independent transformations with the declared prices in Table 1. Kernel work is w , protected kernel work is $w + p$, a conditional validation costs v , and an atomic callback's combined entry/exit charge is k . A validation-only callback can be cheaper than the conditional API: it publishes a prepared result on an identity match and leaves the current value unchanged on a mismatch. Hence the cheapest ordinary attempt costs $c = w + \min(v, k)$.

Table 1. A constructed two-job example. With at most one invalidation, the three-mode optimum is 8; the optimum using only fresh protection and cheapest-validation retries is 9. These are declared resource charges.

Job	w	p	v	k	$c = w + \min(v, k)$	Fresh protected cost
0	3	2	0	0	3	5
1	1	1	3	2	3	4

Prepare job 1 outside protection, then use a cached callback. On a match it costs $1 + 2 = 3$; protect job 0 for 5, giving total 8. On a mismatch the callback recomputes for $1 + 2 + (1 + 1) = 5$; no interference allowance remains, so job 0 costs 3, again giving 8. The best two-mode policy costs 9 even with the same cheaper validation-only callback and adaptive job/mode choices. Thus a cap of 8 changes feasibility. Merely replacing zero validation costs in a two-mode model does not recover the optimum.

We establish a serial normal form for a stated atomic-call interface with nonnegative API charges. Programs may retain several speculative preparations, discard them and switch jobs between completed calls. Protection does not survive a call. Opaque whole kernels, captured immutable predecessor outputs and persistent completion define the batch objective. The theorem identifies an optimum over all causal programs of this interface, rather than over a prescribed ordering or retry threshold. It does not claim optimality for arbitrary concurrent code or exposed, retained locks.

The normal form leaves a finite budgeted game on unfinished job sets. Its direct dynamic program has a state for every remaining budget. We prove that the entire value curve instead has a supporting-line representation using only two input-derived slopes per job. A compiler emits values and runtime choices for all integer budgets, and a separate checker verifies the Bellman equations on finite affine partitions. Binary-encoded large budgets therefore need no budget-by-budget expansion. A mixed-price class reduces further to sorted two-fee packing, with $O(|E| + n \log(n+1))$ construction/checking and a cursor policy of linear size. Outside this compatible class, an NP-complete zero-fee subcase already has one possible failure and two dependency layers.

The operational connection has two levels. For a Java atomic-callback subclass, exact identity semantics give a matching adversary with at most B successful external writes, including when keys share an internal monitor. For Deephaven hierarchical snapshots, source inspection supplies bounds on selected logical operations and an existing public API supplies caller-owned finite refreshes. The latter is an upper-guarantee application of the compiler, with additional update-phase information that can improve the game's value. We report this boundary explicitly.

Budgeted dynamic programming, reflected recurrences and input-slope envelopes are established techniques [7, 26, 30]. The contribution is their justified specialization to charged atomic calls: the third mode and normal-form potential, actual-write correspondence, finite slope-basis closure, and an exact charged-price reduction that removes both budget and subset enumeration on a stated class. The experiments test these claims and their implementation, without treating sparse comparator gains or authored examples as evidence of deployment frequency.

2 Charged Atomic Calls

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$ form a finite DAG with predecessor set $\text{Pred}(i)$. Examples and implementation IDs start at zero. A job is ready when all its predecessors have completed. Completion records the exact successful immutable output and persists for the batch, even if a later external write changes the corresponding live object. The objective is one successful foreground transformation per job. It is not a simultaneous all-object snapshot or transaction.

Each ready job has a pure, opaque, whole kernel. It receives an own-input identity and the exact completed outputs of its predecessors. Its fixed unprotected work is $w_i > 0$; under protection it costs $w_i + p_i$, $p_i \geq 0$, including predecessor processing. Kernels finish and return their exact result. They cannot be partially evaluated, replaced by incremental computations, prepared before readiness or discharged by external work. A preparation is a capability tied to its input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

A local state is U (no usable current preparation), C (a usable current preparation), or D (completed). Several ready jobs may simultaneously be in C. Initially every job is U. An invalidation changes an own-input identity, preserving captured predecessor outputs. The environment has a global remaining allowance b , initially $B \in \mathbb{N}$. At zero allowance only matching outcomes are legal.

The declared charge $v_i \geq 0$ covers an identity-conditional publication call. The charge $k_i \geq 0$ covers entry and exit of an atomic callback, including a callback that only validates. It may be supplied as $g_i + r_i$. These charges define a resource metric; they are not measurements of API latency. All prices are integers, or rational prices scaled uniformly to integers. Table 2 defines the

available counted operations. A callback holds protection only during that operation and performs no other job's foreground work.

Table 2. Counted atomic-call interface. A mismatch spends one invalidation token. Cached completion always finishes the job; validation-only mismatch retains the current live entry and reports failure.

Operation	Start	Matching outcome/cost	Mismatching outcome/cost
Prepare	U	C, w_i	—
Conditional publication	C	D, v_i	U, v_i
Validation-only callback	C	D, k_i	U, k_i
Fresh callback	U	D, $k_i + w_i + p_i$	—
Cached callback	C	D, k_i	D, $k_i + w_i + p_i$

Discarding a preparation is free. A free inspection is permitted only with a legal outcome that preserves a currently usable preparation; it cannot certify that future interference is exhausted. Programs can select any ready job after any completed call, retain preparations or repeat inspections. There is no bare acquire/release operation, guard-retaining return or cross-job batch callback. All calls finish, and every noncompleting program, including one executing free cycles forever, has infinite objective. The environment resolves outcomes after observing the program's action.

This distinction between a call boundary and a retained guard is substantive. A program with an exposed guard may preserve protection while choosing its next operation. Positive guard/API costs can then defeat simple serial-macro arguments. The charged theorem below covers the atomic-call interface in Table 2. The richer zero-fee interface and known counterexamples outside this boundary are retained in Section 10.

3 A Three-Mode Normal Form

Write S for the unfinished jobs and

$$\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$$

for available jobs. Reachable unfinished sets are upsets. For proofs we extend the game to all subsets, treating absent predecessors as completed. Define

$$\begin{aligned} m_i &= \min(v_i, k_i), & c_i &= w_i + m_i, \\ \delta_i &= k_i - m_i, & P_i &= p_i + \delta_i, \\ s_i &= w_i + p_i. \end{aligned} \tag{1}$$

An unavoidable baseline from an all-unprepared start is $A(S) = \sum_{i \in S} c_i$. Three serial modes remain at boundaries where unfinished jobs have no retained preparation:

- (1) *Protected*: execute a fresh callback, paying $c_i + P_i$, and remove i .
- (2) *Cheap validation*: prepare and invoke the cheaper validation API. A match pays c_i and removes i ; a mismatch pays c_i , leaves i unprepared and spends one token.
- (3) *Cached completion*: prepare and invoke a cached callback. A match pays $c_i + \delta_i$; a mismatch pays $c_i + \delta_i + s_i$ and spends one token. Both remove i .

Subtract the once-paid baseline. The optimal worst excess of these modes satisfies $F(\emptyset, b) = F(S, 0) = 0$ and, for $b > 0$,

$$F(S, b) = \min_{i \in \mathcal{A}(S)} \left\{ \begin{array}{l} P_i + F(S - i, b), \\ \max\{F(S - i, b), c_i + F(S, b - 1)\}, \\ \delta_i + \max\{F(S - i, b), s_i + F(S - i, b - 1)\} \end{array} \right\}, \tag{2}$$

where $S - i = S \setminus \{i\}$. At $b = 0$ cheap validation attains zero excess; the cached option has excess δ_i , without a mismatch branch. Each mode outcome decreases $|S| + b$, so a minimizing deterministic controller finishes within $n + B$ mode invocations. This is worst-path control against an adaptive environment, not expected-cost control against a fixed distribution.

THEOREM 1 (CHARGED ATOMIC-CALL NORMAL FORM). *For the interface of Section 2, the minimum worst cost over all causal programs from the all-unprepared state is*

$$\text{OPT}(J, B) = \sum_{i \in J} c_i + F(J, B).$$

A serial three-mode controller attains it. Programs retaining several preparations or changing jobs between calls cannot improve the value.

PROOF. The three modes implement Table 2 with exactly Equation (2)'s full costs. Their decreasing rank and captured successful outputs give the upper bound.

For the lower bound, let x be any interface state with unfinished set T . Define

$$\Phi(x, b) = \sum_{i: x_i = U} c_i + \sum_{i: x_i = C} m_i + F(T, b). \quad (3)$$

Every cached job was physically ready when prepared, and its predecessors remain completed. It is therefore available in T . We show that every call has a legal outcome with immediate cost plus successor potential at least $\Phi(x, b)$. Terms belonging to unchanged jobs cancel.

Preparation pays $w_i = c_i - m_i$ and replaces the c_i term by m_i , preserving the sum. A validation call has charge $z_i \in \{v_i, k_i\}$. Relative to its cached baseline m_i , success has excess

$$z_i - m_i + F(T - i, b),$$

and mismatch has excess $w_i + z_i + F(T, b - 1)$. Since $z_i \geq m_i$, their maximum dominates the cheap-validation option in Equation (2). At $b = 0$ the matching branch suffices.

A fresh callback pays $c_i + P_i$ and removes i ; its inequality is the protected option. A cached callback, after subtracting its old cached baseline, gives the two excesses

$$\delta_i + F(T - i, b), \quad \delta_i + s_i + F(T - i, b - 1).$$

Their maximum is the cached option. Discarding C increases the potential by w_i ; inspection admits a preserving branch.

Nature selects a witnessing branch after each action, within budget b . Telescoping gives the initial potential as a cost lower bound, covering arbitrary job switching and retained preparations. Noncompletion costs infinity; intermediate potentials need not equal intermediate optima. $\square$

A resource cap M is feasible exactly when $A(J) + F(J, B) \leq M$. Prices and allowance are supplied; restricted or correlated outcomes preserve the upper guarantee but may lower the optimum.

PROPOSITION 2 (AN OPEN COMMON-FEE SEPARATION). *For two independent jobs with work $0 < w \leq W$, common $v_i = k_i = \kappa$, and $p_i = \lambda w_i$, the third mode strictly improves the optimum at $B = 1$ exactly when*

$$\frac{w}{W} < \lambda < \min\left\{\frac{\kappa}{w}, \frac{w + \kappa}{W}\right\}.$$

In this interval the excess improvement is $\min\{\lambda w, \lambda W - w, w + \kappa - \lambda W, \kappa - \lambda w\}$.

PROOF. Write $f_x = \min(\lambda x, x + \kappa)$ for a singleton. The two-mode excess reduces to $T_2 = \min\{\lambda(w + W), \max(f_w, w + \kappa)\}$; the better cached choice is the smaller job, with $Q = \max\{f_w, (1 + \lambda)w\}$. If $\lambda W \leq w$, $\lambda W \geq w + \kappa$, or $\lambda w \geq \kappa$, then $Q \geq T_2$. Otherwise $f_w = \lambda W$ and $T_2 = \min\{\lambda(w + W), w + \kappa\}$. Subtracting Q gives the displayed positive minimum. $\square$

For $(w, W, \kappa) = (1, 2, 2)$ the interval is $1/2 < \lambda < 3/2$; at $\lambda = 1$ total costs are 9 and 10. Thus the separation survives within a family having one shared protection weight and one shared call price. These are resource-price conditions; Section 8 tests, unsuccessfully, a measured mean-time interpretation.

4 Actual External Writes in Java

Repeated stale rejection need not require a new write; writes to completed parents preserve captured outputs.

Use Java 17's `ConcurrentHashMap` with one foreground thread, distinct persistent keys, immutable identity-equal wrappers and fresh, never-reused identities for external writes. Each job updates only its fixed key. Keys are not removed; results are non-null; kernels use own snapshots and captured parents and terminate without exceptions. Conditional replacement accepts identity or equals equality [23, 24]; atomic compute invokes its callback once, under a bin monitor in the inspected source [22, 24]. Different keys may share bins.

Implement Table 2 using `replace(key, expected, result)` and `compute`. Validation-only callbacks return the prepared result on match, otherwise the unchanged entry with a failure flag; cached callbacks recompute on mismatch, and fresh callbacks always compute under protection. Capture permanent milestones locally rather than rereading the map.

Callbacks perform no other-key map operations, other foreground kernels or direct guard work, including on cached matches; no guard survives. Multiple preparations are allowed. Prices charge the specified calls; bin collisions add no unperformed kernel work. Assume scheduling and call completion: finite write counts do not imply fairness.

THEOREM 3 (MATCHING ACTUAL-WRITE ENVIRONMENT). *Under these conditions, with no initial preparations and at most B successful external writes,*

$$\min_P \sup_{E: \# \text{writes}(E) \leq B} \text{Cost}(P, E) = \sum_i c_i + F(J, B).$$

The equality holds for separate or shared internal bins.

PROOF. For any admissible program, classify an unfinished job as C exactly when some retained result has its current identity; otherwise classify it as U. Multiple copies do not create extra capabilities. Set $\beta = B - \# \text{writes}$ already made and use Equation (3) with $b = \beta$.

The lower-bound environment writes only immediately before a selected comparison or callback acquires its monitor. If a current prepared result is about to be validated, it can choose no write or one fresh-identity write. The two cost/potential inequalities are precisely the validation and cached-callback cases in Theorem 1. Each chosen invalidation costs one actual write and changes only that job's identity. The foreground holds no monitor at this boundary, so the selected write can finish even when keys collide.

Fresh preparation on the current identity pays for the U-to-C credit change. Computing from an old identity creates no current capability and pays nonnegative cost. Validating an already stale snapshot is a zero-write rejection, leaving the semantic state unchanged; it is not charged another interference token. A cached-completion callback using a stale snapshot recomputes under protection without requiring another write. If the job is U, its excess above the removed baseline is $k_i + s_i - c_i = P_i$; if another current preparation exists and the job is C, it is $k_i + s_i - m_i = w_i + P_i$. Both use the protected inequality. A stale validation-only callback instead preserves the current live entry and pays a nonnegative charge. Fresh callbacks also use the protected inequality. Discarding the only current preparation increases the potential; copying, retaining stale preparations and free

preserving inspections cannot decrease it. Captured parents remain valid when their live entries change.

Thus every observed operation admits a zero/one-write continuation preserving cost plus potential. At most B writes are inserted, and telescoping gives the lower bound. Shared-bin protection ends at each call boundary; the no-other-job-work rule prevents it from doing additional foreground work for free.

For the upper bound, the emitted serial controller rereads after every failed cheap validation. Its read-to-validation intervals are disjoint. Every failure requires an external write in its interval, so failures inject into distinct writes. A mismatching cached completion likewise consumes an interval containing a write and then removes its job. There are at most B such outcomes. The controller's remaining allowance may exceed the actual remaining-write count, which is conservative. The three modes therefore meet the compiled upper bound in either layout. $\square$

Operational accounting. The native test kernels fold captured predecessor seeds and every entry in an immutable array using fixed arithmetic. For integer w_i , choose length $8w_i - |\text{Pred}(i)| > 0$ on the selected inputs. Parent and array folds count $8w_i$ units. Protected kernel execution adds $8p_i$, identity-conditional calls add $8v_i$, and atomic calls add $8k_i$, including mismatches. This scales the abstract objective uniformly by eight. Allocation, monitor waiting, map/API instructions outside these declared charges, initialization and background work are excluded. The experiment does not estimate those costs from timings.

5 All-Budget Compilation

Direct evaluation requires up to $(B + 1)N$ states, $N \leq 2^n$. Our binary-input construction removes budget enumeration; arbitrary DAGs still retain the ideal dimension.

5.1 Reducing the Self-Referential Choice

LEMMA 4 (SUBSET MONOTONICITY AND CHEAPEST VALIDATION). *If $T \subseteq S$, then $F(T, b) \leq F(S, b)$. For any minimum- c_i job $i \in \mathcal{A}(S)$, its cheap-validation option is no larger than that of any other available job.*

PROOF. For monotonicity, simulate a policy for S , completing omitted jobs without invalidation and with nonnegative excess. Relay retained jobs' actions and outcomes. Every virtually available retained job is physically available; omitted jobs spend no token. Physical cost never exceeds virtual cost. Equivalently, induction on $|S|$ and b compares corresponding terms of Equation (2).

At $b = 0$ all cheap-validation values are zero. For $b > 0$, let $j \neq i$ be available. It remains available after removing i . Choosing cheap validation of j in $S - i$ and using monotonicity gives

$$F(S - i, b) \leq \max\{F(S - \{i, j\}, b), c_j + F(S - i, b - 1)\} \leq \max\{F(S - j, b), c_j + F(S, b - 1)\}.$$

The other branch $c_i + F(S, b - 1)$ is also bounded by the last maximum. Their maximum is the cheap-validation value for i . $\square$

For a child curve f with $f(0) = 0$, define the insertion operator

$$(I_s f)(0) = 0, \quad (I_s f)(b) = \max\{f(b), s + f(b - 1)\}, \quad b \geq 1.$$

Fix a cheapest available i , put $g(b) = F(S - i, b)$, $c = c_i$, and

$$h(b) = \min_{j \in \mathcal{A}(S)} \{P_j + F(S - j, b), \delta_j + I_{s_j} F(S - j, \cdot)(b)\}. \quad (4)$$

At zero, $h(0) = \min_j \{P_j, \delta_j\}$ is allowed to be positive. Monotonicity and feasible completing modes give $g(b) \leq F(S, b) \leq h(b)$. Equation (2) becomes

$$f_0 = 0, \quad f_b = \min\{h(b), \max[g(b), c + f_{b-1}]\}, \quad b > 0. \quad (5)$$

This is a specialization of a reflected recurrence [26], not a new general dynamic-programming principle. Its particular child curves permit the following closed form.

LEMMA 5 (CONCAVE FLOOR AND CAP). *Suppose g, h are nonnegative, nondecreasing and discrete concave, $g(0) = 0$, and $h \geq g$. Define*

$$D(0) = 0, \quad D(b) = \sum_{t=1}^b \max\{c, \Delta g(t)\}, \quad c > 0.$$

Then Equation (5) has solution $f_b = \min\{h(b), D(b)\}$, which is nondecreasing and discrete concave.

PROOF. Since $\Delta g(t) - c$ is nonincreasing,

$$D(b) = \max_{0 \leq t \leq b} \{g(t) + c(b-t)\} = \max\{g(b), c + D(b-1)\}.$$

Induct on b . If $h(b-1) \geq D(b-1)$, substitute this identity directly. Otherwise choose a maximizer t for $D(b-1)$. Because $h(b-1) \geq g(b-1)$, the strict inequality requires $t < b-1$. From $h(t) \geq g(t)$,

$$h(b-1) - h(t) < c(b-1-t).$$

Concavity makes the next difference of h smaller than c . Thus the recurrence selects $h(b) < c + h(b-1)$. The same block inequality for g makes $D(b) = D(b-1) + c > h(b)$. This proves the formula in both cases, including the origin. Flooring preserves nonincreasing nonnegative differences. Pointwise minima preserve nondecreasingness and discrete concavity: minimize $2u_b \geq u_{b-1} + u_{b+1} \geq \min_u u_{b-1} + \min_u u_{b+1}$ over the candidate curves. $\square$

5.2 Integer Insertion and Supporting Slopes

We prove the cached maximum's concavity through its marginal structure.

LEMMA 6 (INTEGER MARGINAL INSERTION). *Let a bounded curve with $f(0) = 0$ have a nonnegative supporting representation $f(b) = \min_{d \in D} (db + q_d)$ for all integer $b \geq 0$, with finite $D \subseteq \mathbb{R}_{\geq 0}$ and $0 \in D$. For $s > 0$, define*

$$Q_s = \max_{b \in \mathbb{N}} \{f(b) - sb\}.$$

Then

$$I_s f(b) = \min \left\{ sb + Q_s, \min_{d \in D} (db + q_d + (s-d)_+) \right\}. \quad (6)$$

Its marginal sequence is that of f with one additional marginal s inserted in decreasing order.

PROOF. The marginals of f are nonincreasing and eventually zero. Let t be the number at least s . For $b \leq t$, $I_s f(b) = f(b)$; for $b > t$, it equals $s + f(b-1)$. This proves insertion and gives $Q_s = f(t) - st$.

Every line on the right of Equation (6) majorizes this inserted curve. Before t , unchanged slopes at least s already majorize f ; smaller slopes receive a nonnegative shift. After t , shifting a smaller slope by $s - d$ exactly forms s plus its old value at $b-1$, while a larger slope provides at least the same increase. The contact line $sb + Q_s$ majorizes both portions.

For attainment, an active supporting slope d_b at integer b satisfies

$$\Delta f(b+1) \leq d_b \leq \Delta f(b),$$

where only the first inequality is needed at $b = 0$. At $b < t$ an active line therefore has slope at least s and is unchanged. At $b = t, t + 1$ the contact line attains. At $b \geq t + 2$, an active line at $b - 1$ has slope smaller than s and its shifted version attains. The cases $t = 0$ and $f \equiv 0$ follow from the same contact line. $\square$

The contact is over integers. For $f(b) = \min\{100, b + 1, 3b\}$ and $s = 2$, $I_s f(1) = 2$. Replacing the integer contact 0 by the continuous supporting envelope's supremum $1/2$ gives 2.5 in Equation (6). Computing a real contact and rounding afterwards is not the stated identity.

The floor operation also preserves a finite input-slope basis. If $f(b) = \min_d (db + q_d)$ and $c > 0$, then

$$\sum_{t=1}^b \max\{c, \Delta f(t)\} = \min \left\{ cb + Q_c, \min_{d>c} (db + q_d) \right\}. \quad (7)$$

Let $a = \#\{t \geq 1 : \Delta f(t) > c\}$, the first integer maximizing $f(b) - cb$. The floor equals f through a , and equals $cb + Q_c$ thereafter. At $b < a$ every attaining line has slope above c ; at and after a , each retained larger-slope line stays above the contact line. This definition also handles a plateau with marginal exactly c , or $a = 0$, without special rounding.

THEOREM 7 (ALL-BUDGET BASIS AND ENCODED SIZE). *For every residual set S , $F(S, \cdot)$ is nonnegative, nondecreasing and discrete concave, with eventual value $\sum_{i \in S} P_i$. It has a supporting representation*

$$F(S, b) = \min_{d \in \mathcal{D}_S} (db + q_{S,d}), \quad \mathcal{D}_S \subseteq \{0\} \cup \{c_i, s_i : i \in S\}. \quad (8)$$

There are at most $2|S| + 1$ lines and at most $4|S|$ positive marginal runs. All reachable-state curves and action data can be constructed and checked in $2^n \text{poly}(L)$ bit time, with no iteration through the magnitude of B .

PROOF. Induct on $|S|$, starting with the zero line for the empty set. By Lemma 6, each cached term in h shifts existing intercepts and adds only slope s_j . Protected terms shift intercepts by P_j . Their minimum is a concave supporting envelope. Lemma 4 gives $g \leq F \leq h$ independently of the desired formula, so Lemma 5 applies. Equation (7) adds only the cheapest available c_i . Union the line families and retain the smallest intercept per slope. This proves Equation (8), concavity and nondecreasingness.

Protecting all jobs gives the bound $\sum_i P_i$. A bounded finite lower envelope of nonnegative slopes is eventually constant; let its limit be L_S . By smaller-set induction, sufficiently large budgets in Equation (2) satisfy

$$L_S = \min_{i \in \mathcal{A}(S)} \left\{ P_i + \sum_{j \in S-i} P_j, \quad c_i + L_S, \quad \delta_i + s_i + \sum_{j \in S-i} P_j \right\}.$$

Here $c_i > 0$ and $\delta_i + s_i = P_i + w_i > P_i$, forcing $L_S = \sum_{i \in S} P_i$.

At most $2|S|$ distinct positive slopes give at most that many continuous-envelope changes. Integer sampling adds at most one bridging unit interval per noninteger change, so at most $4|S|$ positive runs suffice. This is a conservative upper bound, not a tightness claim. Summing $4|S| + 1$ over all subsets gives $(2n + 1)2^n$ profile entries, with at most $n2^n$ action identities.

Generated intercepts obey $0 \leq q_{S,d} \leq \sum_{i \in S} (P_i + w_i)$. A protected shift adds P_i ; a cached shift adds at most $\delta_i + s_i = P_i + w_i$; a new contact is bounded by child saturation; flooring retains old intercepts. Contacts require no unbounded search: for profile triples (a, y, d) ,

$$Q_s = \max_{(a, y, d) \in \text{profile}(f)} (y - sa).$$

Each piece is affine and the final constant tail decreases after subtracting sb , so a listed breakpoint attains the maximum. Intercepts, integer slopes and crossing-neighbor positions consequently have polynomial bit length in L . Each state uses polynomially many shifts, contacts and envelope intersections on $O(n)$ lines. Affine-partition checking below has polynomially many such endpoints per state. Enumerating at most 2^n states proves the bit bound. $\square$

5.3 Emitted Policies and Independent Checking

Profiles store (a, y, d) for $y + d(b - a)$ between integer starts, with decreasing positive marginal runs and a constant tail. Both constructors process reachable upsets child-first: one uses floor/cap composition, the other Equations (6)–(7). Neither enumerates budgets.

Runtime selection evaluates Equation (2) using $O(n)$ indexed profiles and a deterministic argmin, with polynomial query cost in n , stored size and $\log(B+1)$. It decrements the allowance on mismatch. At zero budget choose a zero-excess mode: cheap validation qualifies, and cached completion ties when $\delta_i = 0$. Optimal ties need not give pointwise dominance.

The checker imports no constructor. It reconstructs readiness and the reachable domain, validates raw and derived prices, baseline, available-job lists, cheapest-fast IDs, origins, continuity, decreasing slopes and tails, and verifies Bellman equality. Partition at own/child starts and their one-step shifts. Within each interval all atomic expressions are affine. Checking endpoints and floor/ceiling neighbors of all crossings proves equality throughout; check the constant final interval directly. Compare the supporting basis with its emitted curve by the same affine method.

Induction on $|S| + b$ certifies values and the mathematical argmin. Executable selection, arithmetic, parsing and the operational abstraction remain trusted or separately tested. Earlier routing-binding failures remain recorded in the artifact and repaired by RQ1’s field checks. Polynomial-per-state bounds concern generated certificates; arbitrary encodings additionally cost their supplied length. Checking does not establish application-price adequacy.

What a requested-value query can skip. Large b need not require a profile. Classical concave allocation [11] gives the following charged-model bounds:

$$\max_{\substack{b_i \in \mathbb{N} \\ \sum_i b_i \leq b}} \sum_{i \in S} \min(P_i, b_i c_i) \leq F(S, b) \leq \min_{t \in \{0\} \cup \{c_i; i \in S\}} \left(\sum_{c_i > t} P_i + bt \right). \quad (9)$$

For the lower bound, assign private failure budgets b_i . Protected or cached-failure completion pays at least P_i ; any other completion after exhausting its private allowance has already paid $b_i c_i$ in cheap failures. Their total allowance is at most b . For the upper bound, protect jobs with $c_i > t$ and retry the others in any topological order. Their failure costs are at most t . The allocation merges compressed marginal runs: $\lfloor P_i/c_i \rfloor$ copies of c_i , then $P_i \bmod c_i$ if nonzero. It never iterates to b . Equality certifies a value directly, and $b \geq \sum_i \lfloor P_i/c_i \rfloor$ guarantees the constant tail $\sum_i P_i$. Our requested-value comparator uses these tests at every subproblem and separately certifies equality leaves and concave peaks. It returns requested values, whereas the full artifact additionally supplies all-budget profiles and routing data.

6 A Compact Charged Class and the General Barrier

The general compiler handles arbitrary dependencies and prices. A price condition, however, permits a stronger result even with positive call charges and an essential cached mode.

6.1 Reducing Mixed Charges to Two Fees

For a concave suffix f with $f(0) = 0$, write

$$R_{a,q}f(b) = f(b) + \min \left\{ q, \sum_{\ell=1}^b (a - \Delta f(\ell))_+ \right\}. \quad (10)$$

This is the two-fee operator with failure price $a > 0$ and protected premium $q \geq 0$: by Lemma 5, it equals $\min(f + q, L_af)$, where $L_af(b) = \max_{0 \leq j \leq b} \{f(j) + a(b - j)\}$. It is monotone in f .

LEMMA 8 (TWO-FEE ORDERING AND PACKING). *Independent two-fee jobs admit a fixed optimal nondecreasing- a completion order, including against adaptive choices. Composing their R operators in this order gives their value H . Reverse composition fills marginal deficits below the next a : all positive suffix marginals except possibly the last are at least a . Each job adds at most two runs. Every sorted suffix is the root curve capped at its remaining premium sum.*

PROOF. Let G be the optimum restricted to the sorted order. Inserting a protected job i at its sorted position gives $G(S, b) \leq q_i + G(S - i, b)$ without spending a token. Induct on $|S| + b$. The first sorted job has minimum failure price; the proof of Lemma 4, with only protected and cheap modes, makes its cheap option no larger than any other's. The smaller-set and previous-budget values equal G by induction. Thus G is bounded above by every unrestricted Bellman option, while feasibility gives the reverse inequality.

Equation (10) preserves marginals at least a and raises smaller ones toward a using at most q mass. In reverse sorted order, only the final positive singleton can be below a . Raise it first; write any remaining mass as $ha + r$, $0 \leq r < a$, and append h copies of a and one r if nonzero. Constantly many run operations suffice. Removing the added tail mass recovers the previous suffix, proving the cap identity. $\square$

THEOREM 9 (EXACT MIXED-PRICE REDUCTION). *Suppose $p_i \delta_i = 0$ for every job, with the original prices from Equation (1), and set*

$$t_i = \begin{cases} \min(c_i, s_i), & \delta_i = 0, \\ c_i, & \delta_i > 0 \quad (\text{hence } p_i = 0). \end{cases} \quad (11)$$

For independent jobs, the charged excess $F(S, b)$ equals the two-fee optimum $H(S, b)$ with prices (t_i, P_i) at every integer budget. A fixed nondecreasing- t completion order is optimal. The same equality holds for a DAG satisfying $t_u \leq t_v$ on every edge $u \rightarrow v$, with ties ordered topologically. The total charge remains $\sum_{i \in S} c_i + H(S, b)$.

PROOF. Marginal insertion (Lemma 6) commutes with deficit filling when $s \geq a$:

$$R_{a,q}I_s f = I_s R_{a,q} f. \quad (12)$$

Indeed, insertion of s precedes every marginal below a and adds no deficit; $R_{a,q}$ changes only those later marginals, raising them at most to a .

If f is a sorted suffix with effective costs at least t , $s \geq t$ and $q < s$, then $R_{t,q}f \leq I_s f$. Before the final partial marginal the deficit is zero. At that marginal the increment is at most $t - \Delta f(b) \leq s - \Delta f(b)$; after saturation it is at most $q < s$. These cases also show $R_{s,q}f = \min(f + q, I_s f)$ when $t = s$.

For $\delta_i > 0$, $p_i = 0$, so cached completion is dominated: $\delta_i + I_{s_i} f \geq P_i + f$. The ordered charged operator is exactly R_{c_i, P_i} . For $\delta_i = 0$, $P_i = p_i < s_i$, and it is $\min(f + P_i, L_{c_i} f, I_{s_i} f)$. The preceding suffix inequalities make this R_{t_i, P_i} , both when $t_i = c_i$ and when $t_i = s_i$. Thus the sorted charged policy attains H .

For a matching lower bound, the two-fee Bellman equation bounds $H(S, b)$ by $P_i + H(S - i, b)$ and by $\max\{H(S - i, b), c_i + H(S, b - 1)\}$, since $t_i \leq c_i$. These cover protected and cheap choices.

Positive- δ_i cached choices are covered by protection dominance. For a zero- δ_i job at any sorted position, write $H(S) = AR_{t_i, P_i}f$ and $H(S - i) = Af$. Every operator in the preceding composition A has cost at most $t_i \leq s_i$. Monotonicity, the suffix inequality and Equation (12) give

$$H(S) \leq AI_{s_i}f = I_{s_i}Af = I_{s_i}H(S - i).$$

Hence every original charged Bellman option is at least H ; induction on $|S| + b$ proves the lower bound against adaptive choices. Finally, removing DAG edges gives an independent lower bound, and a compatible sorted topological policy attains it legally. $\square$

The class mixes $k_i \leq v_i$ jobs with zero-premium jobs. For Proposition 2, $t_i = \min(w_i + \kappa, (1 + \lambda)w_i)$ increases with w_i , so work order suffices. Effective prices preserve cached choices; the total-cost baseline remains $\sum c_i$.

6.2 A Checked Cursor Policy

Minimum- t Kahn traversal finds a sorted topological order when the edge condition holds. Apply Lemma 8's packing to (t_i, P_i) , retaining the original charged input. For sorted suffix S_k , root curve H and $Z_k = \sum_{i \in S_k} P_i$, store

$$F(S_k, b) = \min\{H(b), Z_k\}. \quad (13)$$

The artifact uses $O(n)$ entries beyond the graph. It covers the $n + 1$ suffix states reached by its own policy, whereas the general artifact covers every reachable unfinished set.

The separate checker recomputes the reduction, original baseline and order. It validates the root's zero origin, positive integer run lengths, decreasing positive marginal heights, implicit zero tail and total mass $H(\infty) = Z_0$, then checks the two-fee Bellman equations without importing the constructor. For adjacent caps $z = Z_k$, $a = Z_{k+1}$, let u and v be the first budgets where H reaches a and z . Check the uncapped prefix, the boundary u , the constant marginal t_i between u and v , and saturation at v ; concavity extends marginal endpoint checks across each interval. Zero premiums require identical adjacent caps. The terminal cap is zero. These conditions verify all budgets by suffix induction; Theorem 9 supplies the charged/global-optimality reduction. Additional original-mode boundary checks are consistency controls, not a substitute for that proof.

Construction takes $O(|E| + n \log(n + 1))$ arithmetic operations. Checking a supplied root with r runs takes $O(|E| + n \log(r + 2) + r)$, plus its encoding length. Generated roots have $r \leq 2n$, giving the same asymptotic bound. Binary operand lengths still count. Prefix lengths/areas give $O(\log(n + 1))$ value queries. At a cursor, evaluate the three original branches in Equation (2) for its fixed next job using adjacent caps. Cheap failure retains the cursor; cached failure spends one token and advances it. All paid charges remain original. Inputs failing the price or edge condition use the general charged compiler.

These are sufficient conditions. With independent $w = (1, 1, 1, 2)$, $p_i = w_i$, common $v = 5$, $k = 6$, and $B = 1$, ascending work order costs 31, while order $(2, 1, 1, 1)$ and the adaptive optimum cost 30; here $p_i \delta_i > 0$. Even with $\delta_i = 0$, sorting by c_i can fail: $(w, p, k, v) = (5, 6, 5, 8), (5, 4, 6, 8)$ gives $c = (10, 11)$ but $t = (10, 9)$, and $B = 1$ totals 31 versus 30. Both refutations remain in the artifact.

6.3 Hardness Outside the Compatible Class

Set $v_i = k_i = 0$. Then $c_i = w_i$, $\delta_i = 0$, $P_i = p_i$, and the cached mode is dominated by a cheap attempt followed by immediate protection on failure. Equation (2) reduces to

$$V(S, b) = \min_i \{p_i + V(S - i, b), \max[V(S - i, b), c_i + V(S, b - 1)]\}.$$

At $B = 1$, a topological order π and the protected jobs H on its no-failure path have worst excess

$$C(\pi, H) = \max \left\{ \sum_{j \in H} p_j, \max_{i \notin H} \left(c_i + \sum_{j \in H: j \prec_{\pi} i} p_j \right) \right\}. \quad (14)$$

After the one possible failure, every remaining job finishes without excess. Minimizing this expression is exact, so an initially fixed order suffices at one failure.

THEOREM 10 (SINGLE-FAILURE HARDNESS). *The zero-fee subcase of deciding $F(J, 1) \leq K$ is NP-complete with at most two DAG layers, maximum indegree two, unit premiums and at most two distinct positive failure prices.*

PROOF. An order and its mode bits can be checked in polynomial time by Equation (14). We reduce from CLIQUE [32]. After trivial cases, let $k \geq 2$, $m \geq \ell = \binom{k}{2}$; add an isolated vertex so the graph has $n > k$ vertices. Set $K = n + m - \ell > k$. Every vertex becomes a job with $(c, p) = (K + 1, 1)$; every edge becomes a job with $(c, p) = (K - k, 1)$ depending on its two endpoints.

A k -clique permits protecting its k vertices, running its ℓ edges fast, and protecting the remainder if no failure occurs. Failure costs $k + K - k = K$, and no-failure premium is $n + m - \ell = K$.

Conversely, value at most K forces all vertices on the no-failure path to be protected, since one vertex failure costs $K + 1$. Every fast edge follows at most k protected jobs. Its endpoints belong to the same first k protected vertices; intervening protected edges cannot enlarge that set. If f edges run fast, no-failure premium $n + m - f \leq K$ requires $f \geq \ell$. At most k vertices contain at most ℓ distinct edges, so they contain a k -clique. Numbers are bounded by $n + m + 1$. $\square$

This restricted NP-completeness result does not establish membership in NP for arbitrary binary B , nor necessity of ideal enumeration. A cover by w chains of lengths n_ℓ bounds the ideal count by $\prod_\ell (n_\ell + 1)$; polynomial for fixed w (XP, not an FPT claim).

Even in the zero-fee subcase, dependencies can require adaptive ordering. Jobs with $c = (3, 7, 5, 1)$, $p = (2, 2, 6, 2)$ and edges $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 3$ cost 24 versus 26 at $B = 2$. After fast job 0 succeeds, the optimum protects job 1 with two tokens left but starts job 2 fast with one token left. Every fixed order receives optimal adaptive modes and knows the initial budget. This is a constructed separation, not a typical-case benefit.

7 A Source-Derived Hierarchical Snapshot Case

Deephaven retries optimistic hierarchical snapshots and can fall back to shared-lock execution. The inspected server API extracts key directives (K), then materializes an ordered viewport and expanded size (V). Its ordinary Web client uploads static keys, so K does not retry. Our two-refreshing-stage configuration is an authored use of the supported server API [3].

A source-derived resource bound. At commit 6367313a [3], fix a 31-row balanced hierarchy, immutable ID/Parent columns, directive keys 0, 1 and 2, a 16-row single-range viewport and four output columns. Operation-entry counters give $W_K = 4$, $W_V = 104$: the latter contains at most four directive links, four parent lookups, zero unlinks, 32 node operations and 64 output-cell operations. Earlier studies send same-valued Actions; a later fixed study alternates node 1 between Contract and ExpandAll, keeps the other two expanded and changes payloads.

These bounds include mid-body updates. Fixed keys avoid insertion/rehashing. Each partition is one compact range whose iterator retains its bounds when previous-value references advance. Actions select the full tree or its contracted subtree; payloads do not change traversal lengths. Directives add no ancestor or remove a linked child. Expansion uses 32 node operations including null leaves; contraction uses 17 and totals 89 selected operations. Inconsistency exits without traversal restart. These source conditions, fixed before outcomes, justify the common cap 104.

Unprotected selected operations have unit weight, protected ones weight $1 + \lambda$. Supply zero call charges and $p_i = \lambda W_i$ to the compiler. Shorter/aborted bodies charge their executed prefix. Allocation, setup, waiting, API instructions and unselected operations are excluded: this is neither whole-program work nor CPU time. Bounds come from source paths and fixed shapes, not trace maxima.

The official real-time-tree example uses periodic updates without a finite total bound [4]. We instead use public `EventDrivenUpdateGraph` with at most q caller-owned `requestRefresh` invocations [3, 5]. Each starts a native cycle, runs sources and notifications, and completes. The foreground has no free operation waiting for all future invocations to disappear.

The evaluated update contract. A source latch exposes the interval from `START` to mutation. Releasing it finishes that refresh through notifications and `COMPLETE`. Decisions observe synchronized transitions: serial inconsistent bodies charge distinct `START` or completion boundaries, giving at most $2q$ failures. Shared-lock execution drains an open cycle first. Partially notified interleavings are excluded; native notification state is richer than raw clock parity.

An imported profile for $K \rightarrow V$, $B = 2q$, bounds bodies by the zero-fee value plus $W_K + W_V$. Inconsistency charges its failed-body cap and consumes allowance; success charges actual selected work. The materialized K directives persist for V even if live Action changes. This gives an upper guarantee; the native graph cannot realize every abstract adversarial choice.

At $\lambda = 3$, $q = 2$, the cap is 424. One fast-tie script attains 424 via four full failures over two split cycles. A local rule with the same allowance costs 432: it protects when local failure exposure reaches the premium. Attainability by that policy is not whole-system minimax optimality. A phase-aware mode game drains an open cycle by protection and has value 420, exposing slack in the $2q$ abstraction.

Integration boundary. Policy hooks and counters reject skipped/foreign calls, unexpected exceptions or violated bounds; all normal returns are checked. This is a source-instrumented version. The comparison replica implements the upstream two-attempt rule under the same strict harness, with elapsed-time fallback disabled. The graph uses public refreshes, not test-only cycle controls. The caller must own every refresh and supply q ; thread count, periodic rate or hidden writers do not supply it. Automatic applicability and fee calibration remain separate problems.

8 Implementation and Evaluation

We examine three questions: whether the charged game and emitted policies agree with direct checking and native calls (RQ1); what the source-derived case demonstrates (RQ2); and when the compiler's broader construction cost is justified (RQ3). Inputs are authored and the studies are exploratory. Reused cases are identified, and new input-generation rules are fixed before their outcomes; these are not held-out application populations. Every attempted input, failure, timeout, invalid unit and correction remains recorded.

Experiments use an Apple M2 with 24 GiB, Python 3.10.9 (3.12.14 for later methods) and Homebrew OpenJDK 17.0.19. The retained `ConcurrentHashMap.java` matches the cited source; the binary was not rebuilt. Hashes bind inputs, code, profiles, commands and traces. Separate Python implementations check Java outputs and affine equations. Model/code proposals, source analysis, proof drafting and manuscript preparation used Codex and other assistants. These checks are author-side evidence, not mechanical proofs or independent blind review. No human or simulated-human study was performed.

8.1 RQ1: Charged Values, Routing and Native Calls

The revised general compiler/checker replays 3,183 observed inputs with one to eight jobs and prices up to 128 bits. Direct and supporting-basis constructions agree after serialization/reloading, and the checker verifies every routing/price field and all-budget equation. All succeed under five-second/unit and 300-second/campaign caps; 11 certificate controls have expected outcomes. The compact implementation compares 19,485 inputs (15,727 observed, 3,758 newly generated) against the original direct compiler. All root profiles agree; independent policy charging checks 527,409 outcome branches at budgets 0..4. There are 16,770 compact routes and 2,715 general fallbacks, with 29 expected certificate/runtime controls. Strict numeric and explicit-input validation repairs observed Python equality aliases and a null-input binding defect; the adverse versions remain recorded with 14 CLI controls. These are finite implementation checks.

On 34 observed native inputs, all 2,278 generated policy paths and 816 concurrent-writer runs satisfy the charge cap, mode decisions, immutable output reconstruction and captured-parent checks. Across 340 input–budget–layout replay cells, maximum path cost equals the compiled value. The concurrent runs contain 74 failures and 3,060 successful writes. Four additional native controls include rejection of live-parent substitution and omitted conditional charges. Coverage concerns generated paths, not all Java schedules.

Earlier trace checkers accepted wrong failure epochs, substituted outputs, and then an impossible single-writer log reordering. Their outcomes remain recorded. The current checker binds same-job publication epochs and searches foreground/writer microprograms containing reads, allocation, logs, comparisons, publications, bin locks and submit/join. A separate explicit-state checker replays each feasible sequence. All unchanged 3,094 ordinary and four control records retain expected outcomes; 12 record controls and 11 sequence controls distinguish static rejection, infeasibility and valid executions. Search visits at most 189 program-counter pairs. Both implementations share the source translation: this is existential feasibility under that model, not recovered timestamps or a proof of Java semantics.

The compact adapter changes only the loader and selector. Six of 36 screened native fixtures satisfy its conditions; all 30 inapplicable rows remain recorded. Adding 20 authored mixed-price fixtures gives 26 inputs. All 502 policy-path replays and 104 concurrent runs pass an original-price, fixed-order scalar checker and the unchanged source-model checks. All 208 replay-cell maxima equal the compiled value. Four native, eight record, five sequence and nine parser controls have expected outcomes, including rejection reasons. A mixed fixture with positive δ retains cap 8 versus two-mode 9. Java fixtures have at most five jobs and bounded prices; the scalable implementation uses Python integers. All ordinary studies above have zero failed, invalid, timed-out or unexecuted units.

For Table 1, both layouts and every emitted branch cost 64 native units, eight times the abstract optimum 8. The two-mode comparator’s 72 units are an abstract value, not a separately measured Java policy. Per-input construction, checking and native preparation costs remain in the artifact.

A failed mean-time calibration. JMH 1.37 [1] measures nine complete wrappers: lookup, preparation, both conditional outcomes, both validation-only outcomes, fresh protection and both cached outcomes. Four payload lengths and two layouts give 72 cells, each with two forks, three 200-ms warmups and five 200-ms measurements. All 144 forks and 720 measurements complete in 278.11 seconds without retry or exclusion. A fixed nonnegative fit maps preparation to w , conditional paths to v , validation-only paths to k , and fresh protection to $w + k + p$; cached outcomes are separate validation targets. Only two of eight settings meet the pre-fixed 20% maximum-relative-residual criterion. Every unconstrained residual is negative and every fitted p is zero. Of 1,152 fixed two-job/budget/weight roots, 72 fall in admitted settings and none gains from the third mode;

1,080 remain outside the fitted model. Input reuse, assignments and object lifetimes differ between wrappers, so the residual does not isolate protection overhead. Short warmup and fork differences further limit inference. Measured mean-time benefit remains unestablished.

8.2 RQ2: Imported Bounds and an Existing Public Graph

The prefix/END study has three protection weights, 12 scripts and six policies: all 216 native/trace cells succeed. They contain 153 cycles and 223 inconsistent bodies, but zero prefix aborts: positive-depth ExpandAll bypasses the leaf-null check. Source review rejected a proposed alternate input whose intended abort mechanism was absent. The protected-tie compiler beats the stronger local rule in only three cells (424 versus 432 twice; 420 versus 432 once). It loses six of 36 cells to the two-attempt replica and four to the unknown-budget threshold; the fast-tie policy also loses a cell, 216 versus 116. Full matrices remain recorded.

With $\lambda = 3$, four scripts and three policies, all 12 public-graph native/trace cells succeed: 11 public refreshes yield 19 inconsistent bodies. The unchanged imported profile is 208 bytes. Checks bind row order, immutable structure, consistent payload epochs, body caps and refresh counts. Under the cross-stage script, compiler/local/replica charges are 424/432/632, with refresh counts 2/0/2; without updates all cost 108. Protection can suppress later script cuts, so realized writer workloads are policy-dependent.

Changing Actions extends this to all 12 scripts and three policies: all 36 cells succeed, with 33 public refreshes. Separate DFS reconstruction verifies 28 full-tree and eight contracted results. Two executions retain epoch-1 contracted K while returning epoch-2 payloads after live Action re-expansion; the successful-body ledger binds the captured epoch. The compiler beats/ties the same-budget local rule in 3/9 scripts and the replica in 6/6. One script costs 275/432/542, below compiler body/model caps 320/424.

A stronger checker verifies exact selected-work totals, including normally returned inconsistent bodies, previous-value choice during paused sources, immediate eligible cuts and synchronous refresh blocking. All 36 records pass; 17 corruptions are rejected and four positive controls pass. The old checker missed eight of those corruptions. Ten upstream/MAX-key regressions also pass. An exploratory phase-aware study has 108 successful cells: protection can drain an open cycle, and already-notified sources can use current values while others retain previous values. Thus the $2q$ compilation remains an upper-bound application. Every study retains its full outcomes, source conditions and timing records; none establishes the unobserved abort path or whole-system native minimax optimality.

8.3 RQ3: Construction Cost and Stronger Value Queries

The general charged comparison uses 24 authored inputs: four topologies (chain, four chains, fence, independent), $n \in \{8, 12\}$ and three price families. MIXED is small/heterogeneous; LONG multiplies protection premiums by 2^{32} ; WIDE scales all prices by 2^{96} . Workloads request the first 1, 4 or 12 budgets of (1, 2, 4, 8, 16, 32, 64, 256, 4096, 65536, 2^{32} , 2^{64}). ALL constructs, writes, reloads and checks all-budget artifacts. PEAK memoizes concave peaks and checks requested-value certificates. DP uses two Bellman rows and stops at an all-state fixed point, returning values only. Cold costs include imports through output/checking; caps are 4 seconds/1 GiB.

Initial successes are 63/72 ALL, 62/72 PEAK and 64/72 DP. Eight generic AND-OR units also succeed, totaling 224 units/1,256 roots. After observation, Equation (9) strengthens PEAK/DP: a separate 144-unit/816-root campaign yields 64/72 successes each. Every other unit times out; all shared values agree. Separate 76-input conformance checks compare 532 values/304 generic roots and 11 certificate controls, all expected. Full cold and phase timings remain recorded.

On 14 common all-12 successes, revised PEAK/ALL median time is 0.576 versus initial 7.16; revised DP/ALL is 0.457. These separate campaigns have one sample per unit. Seven LONG full constructions finish while both value methods time out; uniform WIDE scaling preserves budget-axis geometry and gives no such advantage. All nine ALL timeouts are independent 12-job inputs. Thus elementary bounds remove most of the original peak disadvantage, while ideal enumeration/full checking remain costly.

Compilation on the compatible charged class. A separate campaign fixes 42 new inputs with $n \in \{8, 12, 32, 128, 512, 4096, 32768\}$, independent or compatible height-two DAGs, and small, long-premium or uniformly 2^9 -scaled prices. All have $\delta_i = 0$; each edge satisfies the derived t order. Workloads request the first one or all 12 budgets above. PACK constructs the charged cursor artifact, writes/reloads it, checks it and builds the runtime index before querying. ROOT uses the same reduction and packing but returns only a root value curve without independent checking. It is an ablation of the new method, not an independent prior algorithm. ALL, PEAK and DP retain their stated services. PEAK, DP and the point checker use adjacency-based graph validation and grouped elementary-bound intercept scans after sorting.

All 420 cold units terminate in 1,315.17 seconds under eight-second/1-GiB per-unit caps, with no invalid or unstarted unit and no shared-value disagreement. Table 3 includes every outcome. PACK completes all 84 units; its 12 runs at 32,768 jobs take 0.837–1.151 seconds end to end. Its maximum RSS is 122,044,416 bytes and maximum serialized artifact is 8,084,847 bytes, including the input. Checking/indexing dominates those large runs. ROOT completes all 84 with a maximum 0.241 seconds, exposing the additional cost of a checked policy. On common successes, PEAK/PACK and DP/PACK median cold-time ratios are 0.860 and 0.792; a requested value can still be cheaper. All ratios are descriptive single samples.

Table 3. Compact-class campaigns. Each cell is success/timeout/memory limit. A: 84 units per method; B: 36 units per method with $2n$ root runs. All outcomes are retained; returned services differ.

Method	A	B	Returned service
PACK	84/0/0	36/0/0	Checked all-budget cursor policy
ROOT	84/0/0	36/0/0	Same-reduction values, no checker
ALL	22/56/6	0/30/6	Checked all reachable unfinished sets
PEAK	34/50/0	18/6/12	Checked requested root values
DP	34/46/4	18/15/3	Requested values only

Those large-job curves have only 40 runs. A second campaign fixes 18 mixed-price inputs: $n \in \{512, 4096, 32768\}$, the same two topologies and three numerical scales. Their root curves have exactly $2n$ runs; long instances contain 2^{32} -length plateaus. The last four query budgets become $(2^{32} - 1, 2^{32}, 2^{32} + 1, 2^{64})$. All 180 cold units finish recording under the same caps: PACK/ROOT succeed in all 36 each, while PEAK/DP succeed only for the 18 one-budget workloads (Table 3). All 36 PACK profiles equal the pre-fixed run sequence; every successful value matches its pre-fixed closed form. At 32,768 jobs, complete PACK runs take 1.333–1.603 seconds with 65,536 root runs; maximum RSS is 152,911,872 bytes. Median PEAK/PACK and DP/PACK ratios on common successes are 0.301 and 0.206. Final aggregation encountered a variable-shadowing error after every raw/process record and analytic check had been saved; a separate post hoc script recovers the full denominator without rerunning measurements. Neither campaign has a value disagreement, invalid unit or unstarted unit. These are structural scale tests, not an application distribution.

A retained 874-input charged-order exploration compares 7,866 roots at budgets 0..8. All succeed; only the known zero-fee four-job case gains from adaptive order (24 versus 26). None of 840 new independent inputs does, although post hoc analysis finds 16 where the best c -sorted order is suboptimal. These finite observations do not extend Theorem 9 beyond its conditions.

Earlier zero-fee evidence remains separate: 4,232 primitive-price inputs, 19,644 native paths on 48 DAGs, and adaptive-order gains in only two of 192 roots, both from one input. At $B = 1$, CP-SAT completes 78 roots, screened DP 68 and full construction 53; the others time out. The artifact retains all construction/checking costs, failures, invalids and unstarted checks. New charged results do not replace those adverse comparisons.

9 Related Work

Quantitative synchronization synthesis. Černý et al. [17] already choose optimistic retries, guards and caches. Their Theorem 6 preserves limit-average values through matched distributions/weights; an erratum supersedes the original hardness claim. Our potential establishes initial worst-total-cost equality for charged calls and actual writes. Their later work [18], Sections 6.1–6.2, optimizes protected-statement counts and profiles block costs under contention and lock acquisition. Our declared resources lack that performance calibration. Weighted reachability already expresses optimal worst fault cost [6]; resilience/cost synthesis and general game solvers are established [8, 9].

Reduction and robust dynamic programming. Gangamreddypalli et al. [19] preserve failures and reach each original successful final state or replace it by failure. Our calls do not commute unconditionally with same-key writes; the potential is not an exact intermediate-state quotient. A cost-preserving reduction remains unestablished. Mannor et al. [7], Section 4/Theorem 5, provide finite global-deviation-budget DP: unfinished upsets, completing nominal branches and token-consuming failures instantiate it with negated cost and horizon $n + B$. Postek et al. [29] supply exact finite-candidate DP, state expansion and causal budgeted routing. These express the finite game; the normal form and all-budget closure need additional arguments.

The public zero-fee package [33] is an earlier snapshot of this same research project. Its hardness, concave floor/cap lemma and two-fee ordering/packing are reused here. The charged additions are the atomic-call potential, second supporting slope, and mixed-price reduction; the artifact maps the results individually.

Envelopes, reflection and allocation. Burdzy et al. [26], Equations (2.12)–(2.13)/Theorem 2.6, give the general reflected recurrence; our floor/cap lemma specializes it to concavity. Marginal insertion is elementary. Bertsimas and Sim [30], Theorem 3/Equations (15)–(19), already give an input-deviation slope envelope $Z(\Gamma) = \min_{\ell} \{\Gamma d_{\ell} + F_{\ell}\}$. Such envelopes alone are not new. Here Equations (6)–(8) close the operations for repeated causal failures, two completing modes and dependent job choices. Reduction to a fixed binary robust set remains unestablished. Concave allocation [11, 12] supplies lower bounds; decision-dependent information and controllable uncertainty are broader settings [14, 15].

Scheduling and hardening. At one failure, the zero-fee problem couples processing/tail modes $(p_i, 0)$ and $(0, c_i)$. Fixed modes use Lawler’s precedence sequencing [27]; our hardness concerns selecting modes. Izosimov et al. [20] activate DAG schedules under bounded failure histories with signalling/transparency constraints; An and Yang [10] synthesize stochastic hardening under fixed priorities. We omit those constraints. Chrobak et al. [13], Lemma 4/Theorem 5, concentrate faults on a critical job and compute prescribed-order latest completion in $O(n)$ independently of the fault bound. Their model mandates retry; ours chooses priced protection/cached completion and returns all-budget counted-work policies. Bold and Goerigk [31] use $\Gamma + 1$ DAG layers and a compact

MIP after duration observation. Their activities complete despite delay; our failure leaves a job unfinished before another causal choice.

Ordering and rent-or-buy. Heterogeneous thresholds and ordering appear in ski rental [21, 25]. The direct map $f(H) = \sum_{i \in H} p_i$, $g(H) = \sum_{i \in H} c_i$, $T = B$ gives 6 for $(p, c) = (2, 5), (4, 6)$ at $B = 1$, versus our zero-fee value 5. This excludes only that map: shared interference, mandatory work and causal completion must be matched. At $B = 1$, charged fixed-order excess is $\psi_{\pi_1} \circ \dots \circ \psi_{\pi_n}(0)$ with $\psi_i(x) = \min\{x + P_i, \max(x, c_i), \delta_i + \max(x, s_i)\}$. Kawase et al. [2] establish optimal composition, exchange arguments and tractable/hard piecewise-linear scheduling classes. That scalar view is inherited. Their results do not directly resolve our generally nonconvex/nonconcave maps, and one-budget composition does not establish the all-budget reduction or closure.

10 Limitations and Threats to Validity

The charged theorem requires whole fixed-price kernels, own-input identity changes, captured immutable parents, persistent completion, terminating calls and no retained protection or cross-job foreground work inside callbacks. Partial/incremental evaluation, replenished budgets, structural changes, exceptions and external milestone discharge are outside it. A free certificate that all future writers have finished can reduce the optimum to baseline work, so such an operation cannot be added without analysis. The map itself supplies neither the finite write allowance nor calibrated prices.

The richer zero-fee interface admits individual or batch guard acquisition, retained guards and preparations. Its earlier one-pass/potential argument shows equality with the two-mode game from an all-unprepared boundary. Charging exposed guard operations changes that argument. Earlier positive-cost failed-acquisition experiments retain five strict improvements among 384 inputs, including unrestricted 242 versus macro 243. In a later exposed-guard model, a proposed local rule releasing a failed guard or completing it immediately is refuted by an intermediate-state value 43 versus 45. Full all-unprepared equality for that charged exposed-guard model remains unproved, despite finite agreement. We do not use the atomic-call theorem to close those questions.

The Deephaven guarantee covers selected operations, a fixed hierarchy/directive-key set, the stated Action changes, owned finite refreshes and synchronized boundaries. Ordinary Web use has a static key stage; periodic operation has no supplied finite q . Source notification state distinguishes executions with the same raw clock. The public-graph study establishes an API composition and resource accounting; deployed frequency and exact whole-system minimax optimality remain unknown. Zero observed prefix aborts and policy-conditioned suppression of writer cuts remain part of the result.

All tests are authored and finite, and proofs are not mechanically verified. Certificate checking establishes the declared game's equations; native traces test implementations under recorded schedules. Strong-comparator gains are sparse in some families, and direct methods avoid timeouts incurred by full construction. The API calibration does not supply a measured positive premium or a timing benefit; threshold relaxation after its outcome would not repair that result. No developer-effort reduction or observed human decision improvement was measured.

11 Conclusion

Cached atomic completion can be essential when validation and callback calls are charged. An action-wise potential yields an exact three-mode normal form, with all-budget supporting bases and a linear-size mixed-price cursor representation on compatible DAGs. Conditional actual-write correspondence and a source-derived snapshot application delimit the guarantees that reach native code.

Data Availability

The charged-call and Deephaven supplement is public at anonymous commit 610e415 [34], with fixed inputs, constructors/checkers, native sources, raw traces, complete outcome ledgers, exact replay commands and a theorem-lineage map. The preceding zero-fee package [33] remains unchanged. Oversized input ledgers use lossless compression with decoded hashes. Licensed modified Deephaven sources retain their full license and notice; upstream archives, runtimes, build caches, third-party articles and private correspondence are excluded.

References

- [1] OpenJDK. n.d. *JMH: Java Microbenchmark Harness*, version 1.37. Accessed September 9, 2026. <https://github.com/openjdk/jmh/tree/1.37>.
- [2] Yasushi Kawase, Kazuhisa Makino, and Kento Seimi. 2016. Optimal Composition Ordering Problems for Piecewise Linear Functions. In *ISAAC, LIPIcs* 64, 42:1–42:13. <https://doi.org/10.4230/LIPIcs.ISAAC.2016.42>. Extended version: *Algorithmica* (2018), <https://doi.org/10.1007/s00453-017-0397-y>.
- [3] Deephaven Data Labs. 2026. *Deephaven Core source*, commit 6367313a319b79437d4d2116c77836ad56d5138d. Files `ConstructSnapshot.java`, `HierarchicalTableImpl.java`, `EventDrivenUpdateGraph.java`, and `JsTreeTable.java`. Accessed September 9, 2026. <https://github.com/deephaven/deephaven-core/tree/6367313a319b79437d4d2116c77836ad56d5138d>.
- [4] Deephaven Data Labs. 2026. *tree: Real-time tree table*. Python documentation, version 0.40.7. Accessed September 9, 2026. <https://deephaven.io/core/0.40.7/docs/reference/table-operations/create/tree/>.
- [5] Deephaven Data Labs. 2026. *Execution context*. Official documentation, custom update graphs. Accessed September 9, 2026. <https://deephaven.io/core/docs/conceptual/execution-context/>.
- [6] Emil Dumitrescu, Alain Girault, Hervé Marchand, and Éric Rutten. 2007. *Optimal discrete controller synthesis for the modeling of fault-tolerant distributed systems*. INRIA Research Report RR-6137. https://www.researchgate.net/publication/250079523_Optimal_discrete_controller_synthesis_for_modeling_fault-tolerant_distributed_systems.
- [7] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [8] Daniel Neider, Alexander Weinert, and Martin Zimmermann. 2020. Synthesizing optimally resilient controllers. *Acta Informatica* 57, 195–221. <https://doi.org/10.1007/s00236-019-00345-7>.
- [9] Alexander Weinert. 2019. *Optimality and Resilience in Parity Games*. PhD thesis, Saarland University. Section 6.2. https://finkbeiner.groups.cispa.de/_assets/paper/C89PMq4G.pdf.
- [10] Hyung-Chan An and Hoesek Yang. 2021. Fully Adaptive Stochastic Handling of Soft-Errors in Real-Time Systems. *IEEE Access* 9, 155058–155071. <https://doi.org/10.1109/ACCESS.2021.3128397>.
- [11] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [12] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [13] Marek Chrobak, Mathilde Hurand, and Jiří Sgall. 2009. Algorithms for Testing Fault-Tolerance of Sequenced Jobs. *Journal of Scheduling*. Author version. <https://iuuk.mff.cuni.cz/~sgall/ps/faults.pdf>.
- [14] Phebe Vayanos, Angelos Georgioui, and Han Yu. 2026. Robust Optimization with Decision-Dependent Information Discovery. *Management Science* 72, 2, 1509–1528. <https://doi.org/10.1287/mnsc.2021.00160>.
- [15] Eva Ley, Rebecca Marx, Maximilian Merkert, Tim Niemann, and Sebastian Stiller. 2026. Decision-dependent robust optimization with elementwise controllable uncertainty sets. *OR Spectrum*. <https://doi.org/10.1007/s00291-026-00856-z>.
- [16] PRISM-games developers. 2026. PRISM-games. Official tool documentation. Version 3.2.4 used in the experiments. <https://www.prismmodelchecker.org/games/>.
- [17] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [18] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [19] Namratha Gangamreddypalli, Constantin Enea, and Shaz Qadeer. 2026. Reduction for Structured Concurrent Programs. *arXiv preprint arXiv:2601.13341*. <https://arxiv.org/abs/2601.13341>.

- [20] Viacheslav Izosimov, Paul Pop, Petru Eles, and Zebo Peng. 2012. Scheduling and Optimization of Fault-Tolerant Embedded Systems with Transparency/Performance Trade-Offs. *ACM Transactions on Embedded Computing Systems* 11, 3, Article 61, 35 pages. <https://doi.org/10.1145/2345770.2345773>.
- [21] Bingham Wu, Wei Bao, Dong Yuan, and Bing Zhou. 2022. Competitive Analysis for Multi-Commodity Ski-Rental Problem. In *IJCAI*, 4672–4678. <https://doi.org/10.24963/ijcai.2022/648>.
- [22] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [23] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [24] OpenJDK. 2026. ConcurrentHashMap.java, jdk 17 u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [25] Ziwei Li, Bo Sun, Zhiqiu Zhang, Mohammad Hajiesmaili, Bingham Wu, Lin F. Yang, and Yang Gao. 2025. Combinatorial Ski Rental Problem: Robust and Learning-Augmented Algorithms. In *NeurIPS*. https://papers.nips.cc/paper_files/paper/2025/hash/ae3e173398d6e43fea63cbcc16fbaa98-Abstract-Conference.html.
- [26] Krzysztof Burdzy, Weining Kang, and Kavita Ramanan. 2009. The Skorokhod problem in a time-dependent interval. *Stochastic Processes and their Applications* 119, 2, 428–452. <https://doi.org/10.1016/j.spa.2008.03.001>.
- [27] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>.
- [28] Laurent Perron, Frédéric Didier, and Steven Gay. 2023. The CP-SAT-LP Solver (Invited Talk). In *CP, LIPIcs* 280, 3:1–3:2. <https://doi.org/10.4230/LIPIcs.CP.2023.3>.
- [29] Krzysztof Postek, Ward Romeijnnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. [Online appendix; author code](#).
- [30] Dimitris Bertsimas and Melvyn Sim. 2003. Robust discrete optimization and network flows. *Mathematical Programming* 98, 49–71. <https://doi.org/10.1007/s10107-003-0396-4>.
- [31] Matthew Bold and Marc Goerigk. 2021. A compact reformulation of the two-stage robust resource-constrained project scheduling problem. *Computers & Operations Research*. <https://doi.org/10.1016/j.cor.2021.105232>. [Accepted manuscript](#).
- [32] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9. [Primary paper](#).
- [33] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [34] Anonymous Authors. 2026. *Compiling Optimal Retry Policies with Charged Atomic Calls: Research Artifact*. Version 6 charged supplement, commit 610e415. <https://github.com/research-artifact-archive/research-artifact/tree/610e415b7f7278cd67a7cfd529a7a71d73c4b035>.